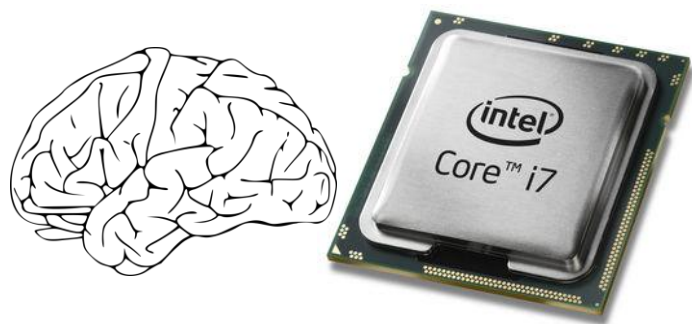


MONITORAMENTO AMBIENTAL SENSORES E MICROCONTROLADORES

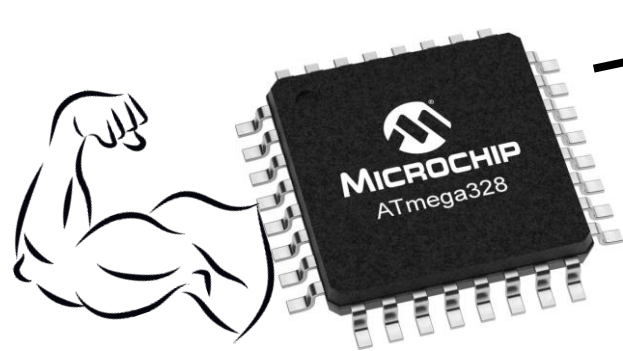
AULA 3

**ESP8266, Sensores Analógicos e Digitais e Datalogger em
Cartão SD**

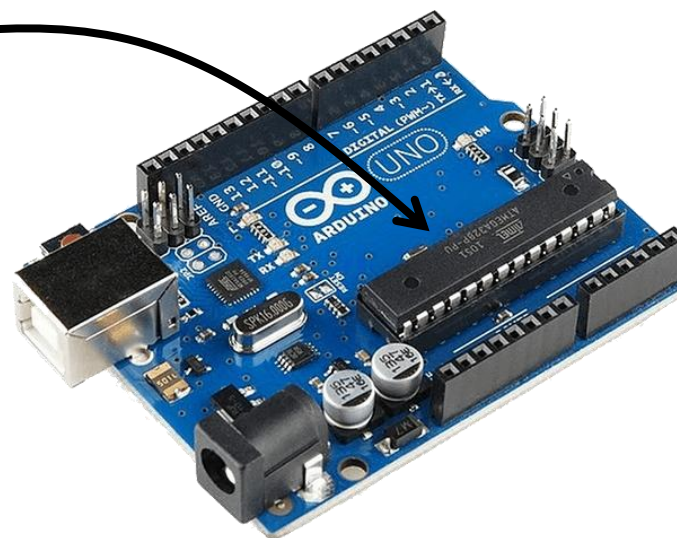
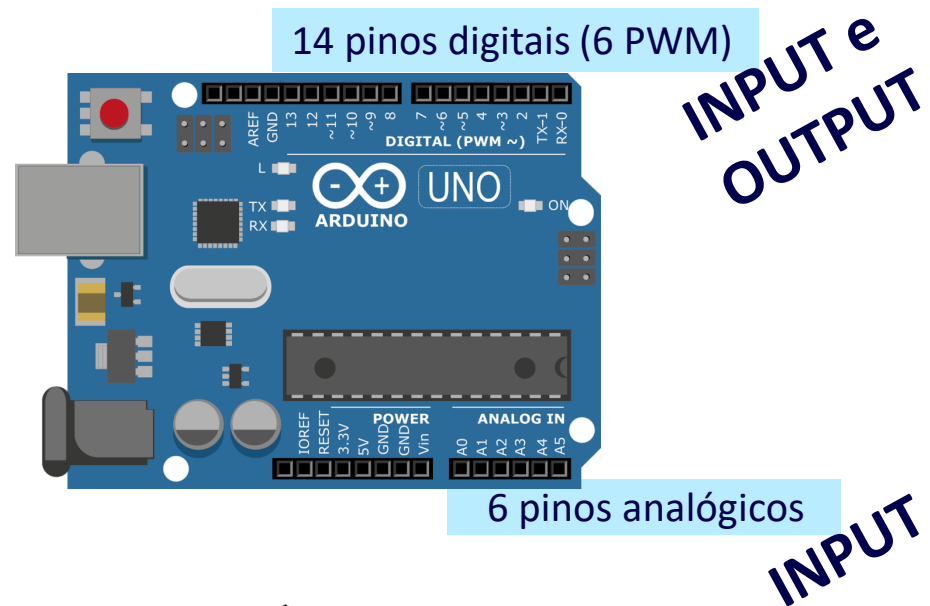


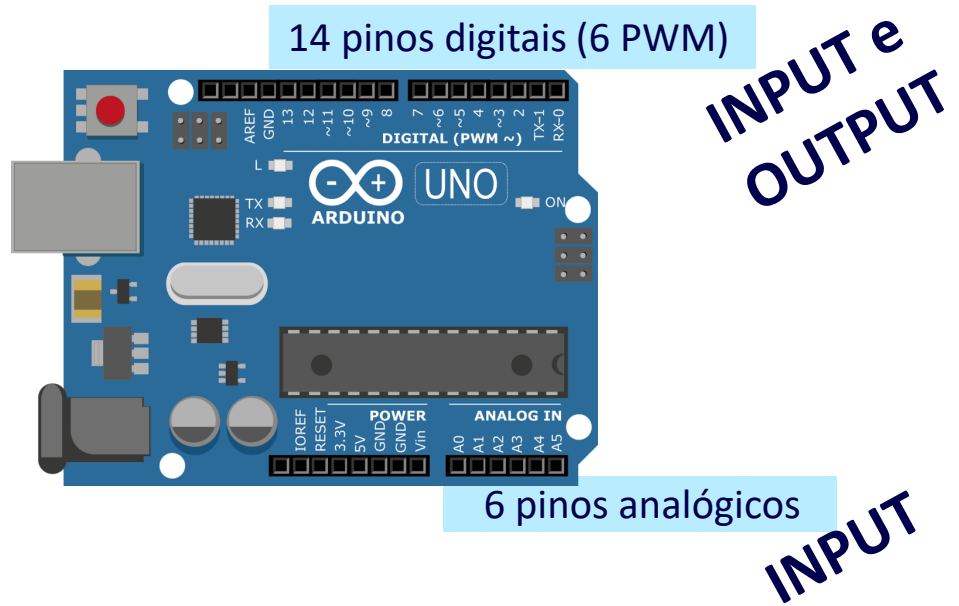


Microprocessador

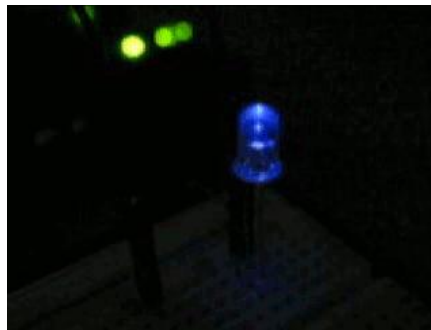


Microcontrolador



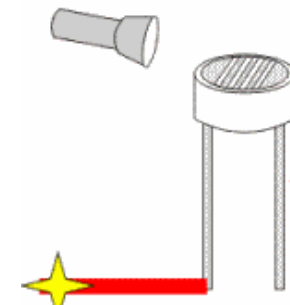


DIGITAL ~ 1 bit

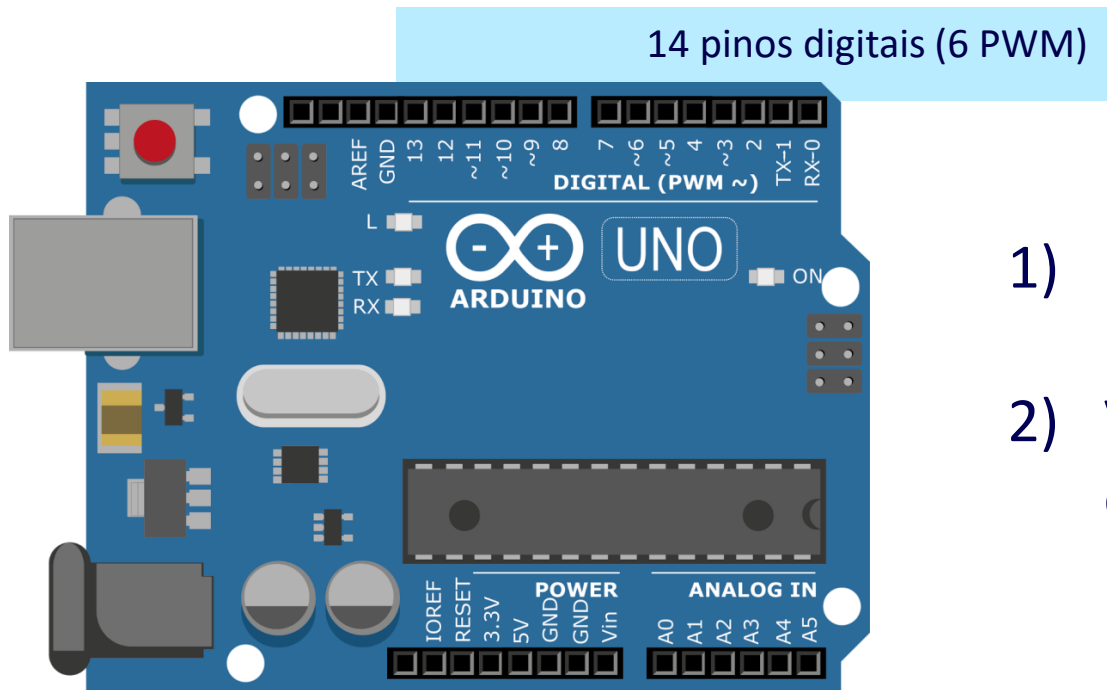


PWM ~ 5 bits

Simular um pino analógico



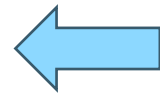
ANALÓGICO ~ 10 bits



- 1) Vamos ver outros sensores analógicos
- 2) Vamos ver como funciona os protocolos de comunicação de um sensor/ dispositivo digital.
> leitor de cartão SD
- 3) Conhecer um outro microcontrolador ESP8266

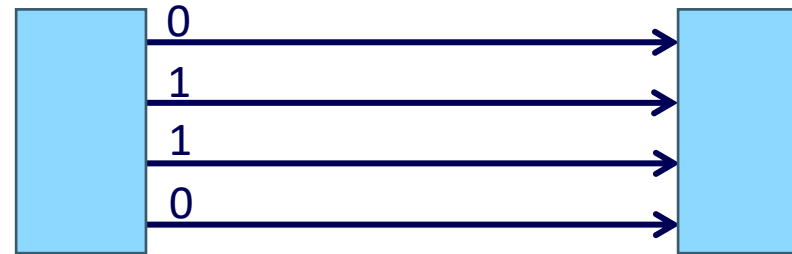
1 – Comunicação serial

Várias formas de comunicação

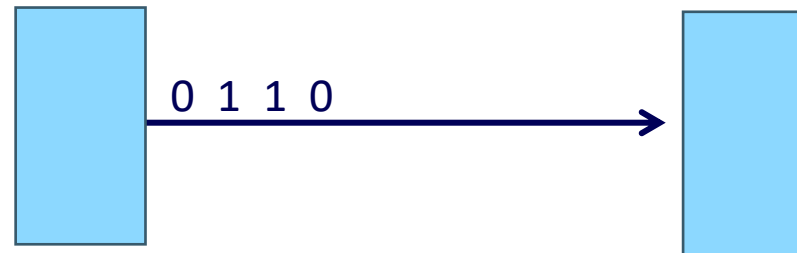


Comunicação serial

Comunicação paralela



Comunicação em série

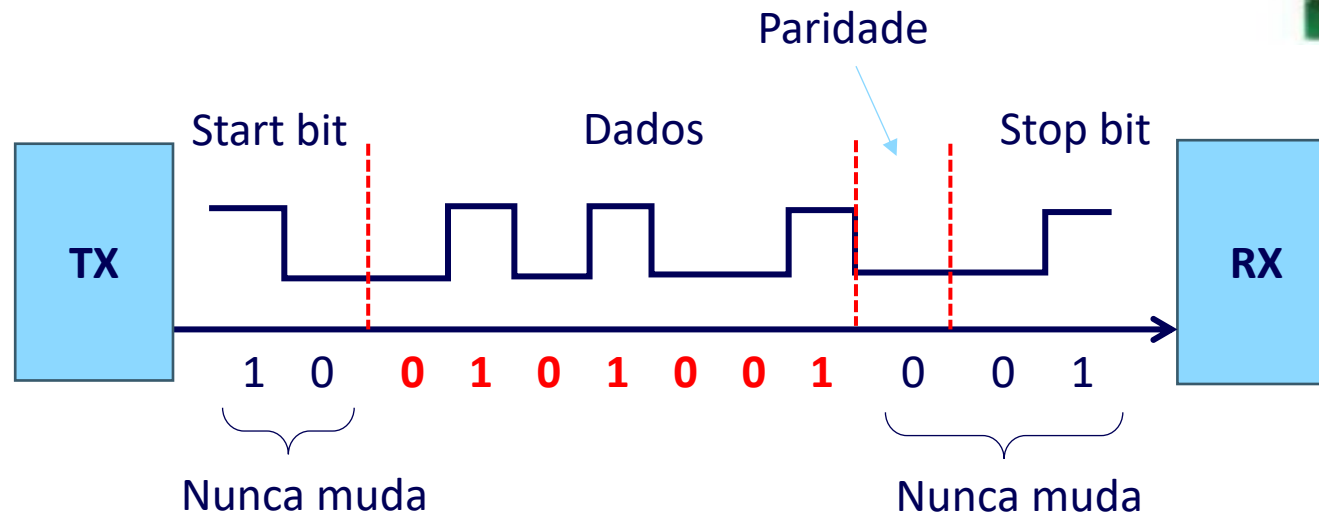


- 1. UART (TX e RX)
- 2. I2C (SCL e DAS)
- 3. SPI (MISO e MOSI)

UART (Universal Asynchronous Reception and Transmission)

Sem sincronismo (sem clock)

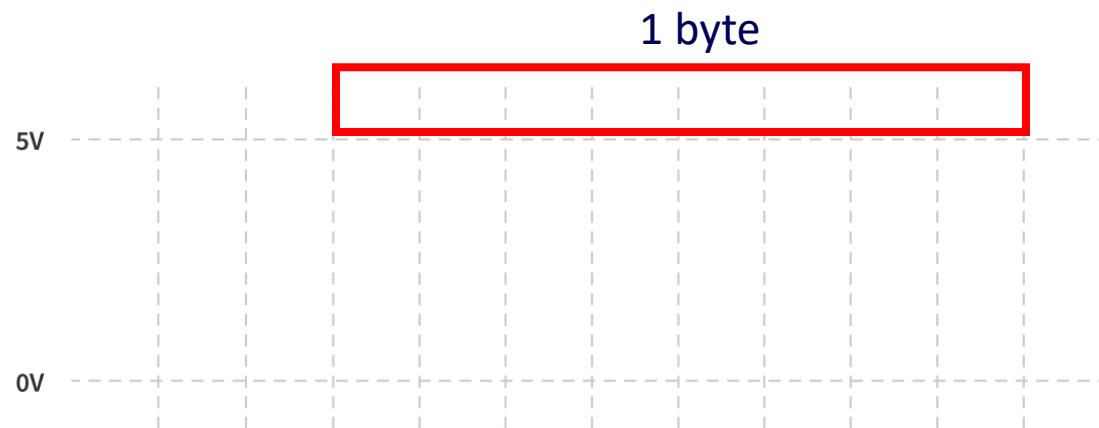
Devem estar configurados na mesma
velocidade de comunicação (BAUD RATE)



UART (Universal Asynchronous Reception and Transmission)

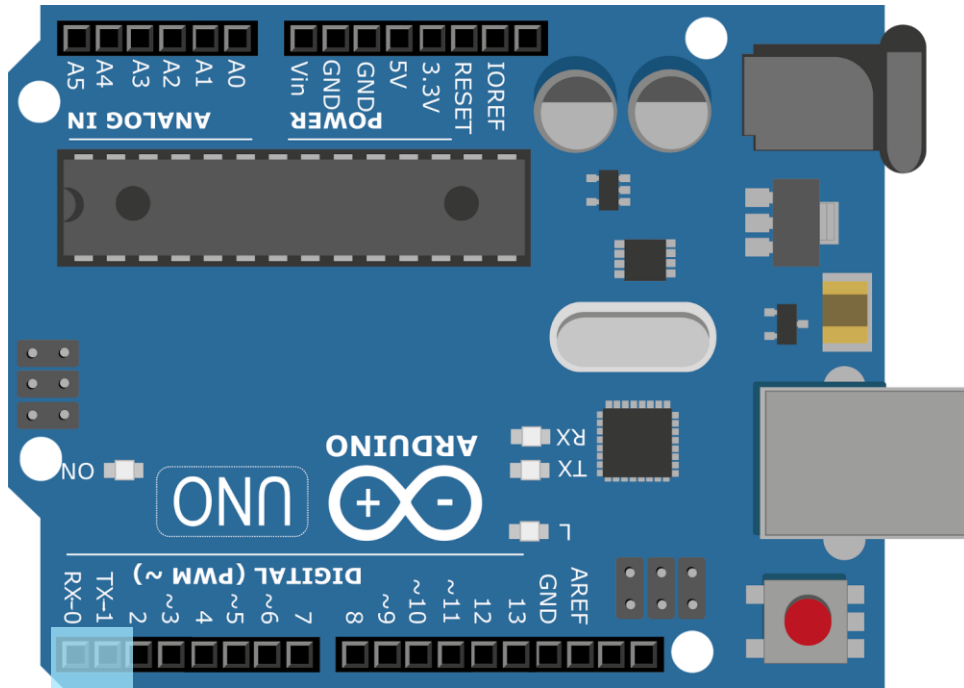
Sem sincronismo (sem clock)

Devem estar configurados na mesma velocidade de comunicação (BAUD RATE)



Data Frame of UART for Transmission of Letter 'H'





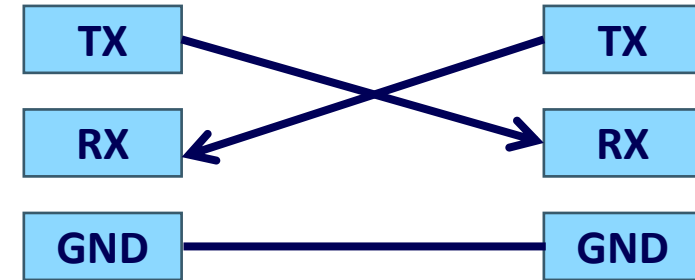
Comunicação serial = Reservado para comunicação via USB
(utilizado para envio de sketch e comunicação com usuário)



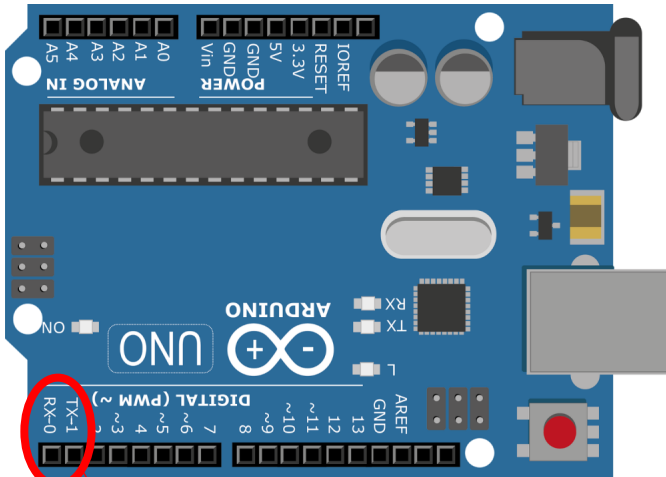
**Não é permitido se conectar com
outros dispositivos quando utilizado
para comunicação serial**



As comunicações UART ocorrem porta a porta.
Para configurar mais portas UART



UART (Universal Asynchronous Reception and Transmission)



As comunicações UART ocorrem porta a porta.
Para configurar mais portas UART



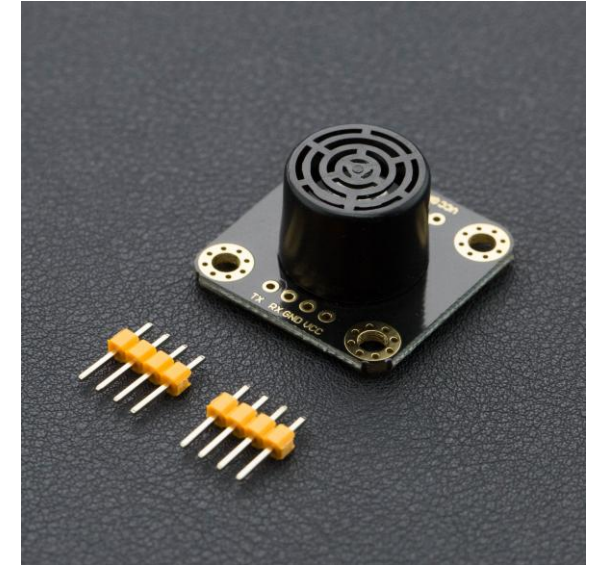
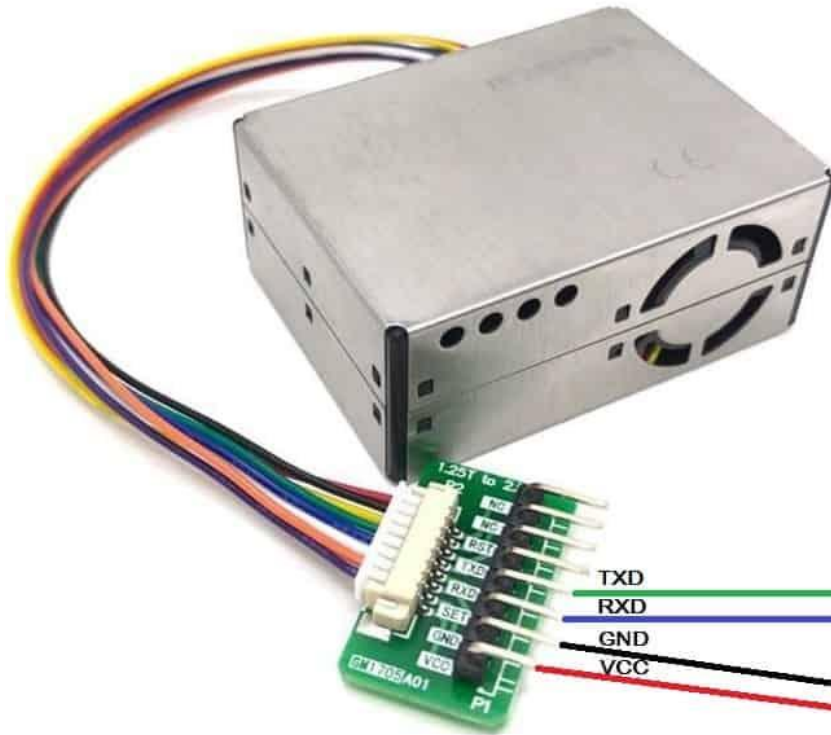
Devem estar configurados na mesma
velocidade de comunicação (BAUD RATE) → Verificar o datasheet

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX, TX

void setup(){
    Serial.begin(9600);
    mySerial.begin(9600);
}

void loop(){
    int value = mySerial.read();
}
```

PM2.5 laser (sensor de material particulado)

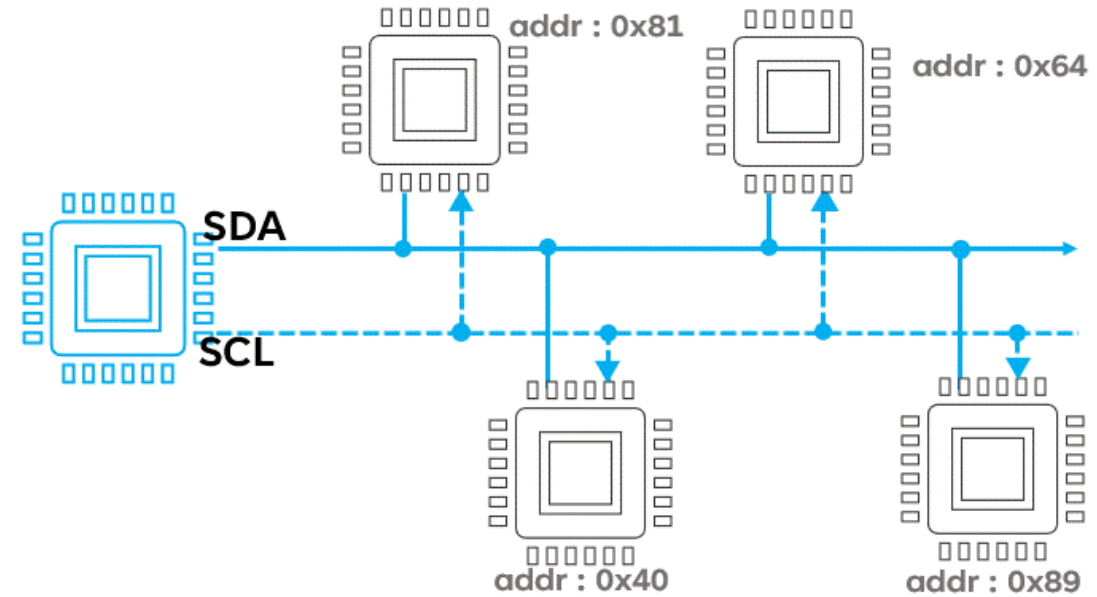


Ultrasonic sensor (sensor de distância)

I2C (Inter-integrated circuit, IIC ou I2C)

2 fios: SCL (clock) e SDA (dados)

Possível combinar até 120 “escravos”
e inclusive ter mais de um “mestre”



Comunicação I2C é mais veloz do que UART

Standard mode: 100 kbit/s

Fast mode: 400 kbit/s

1byte por vez

I2C (Inter-integrated circuit, IIC ou I2C)

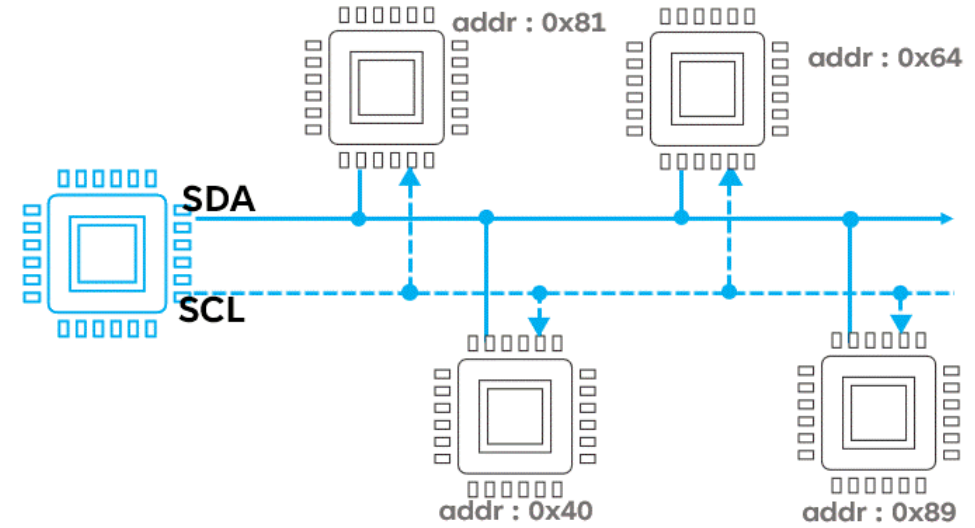
```
Hexadecimal
#include <Wire.h>
#define Sensor_Ad 0x68

void setup() {
    Wire.begin();
    Wire.setClock(400000);
}

void loop(){

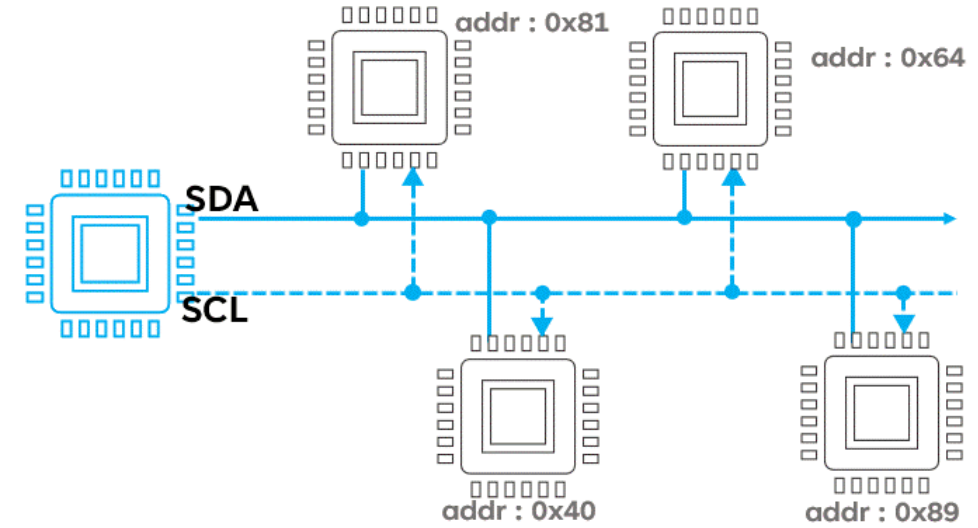
    Wire.beginTransmission(Sensor_ad);
    Wire.write(0x3B);
    Wire.endTransmission(false);

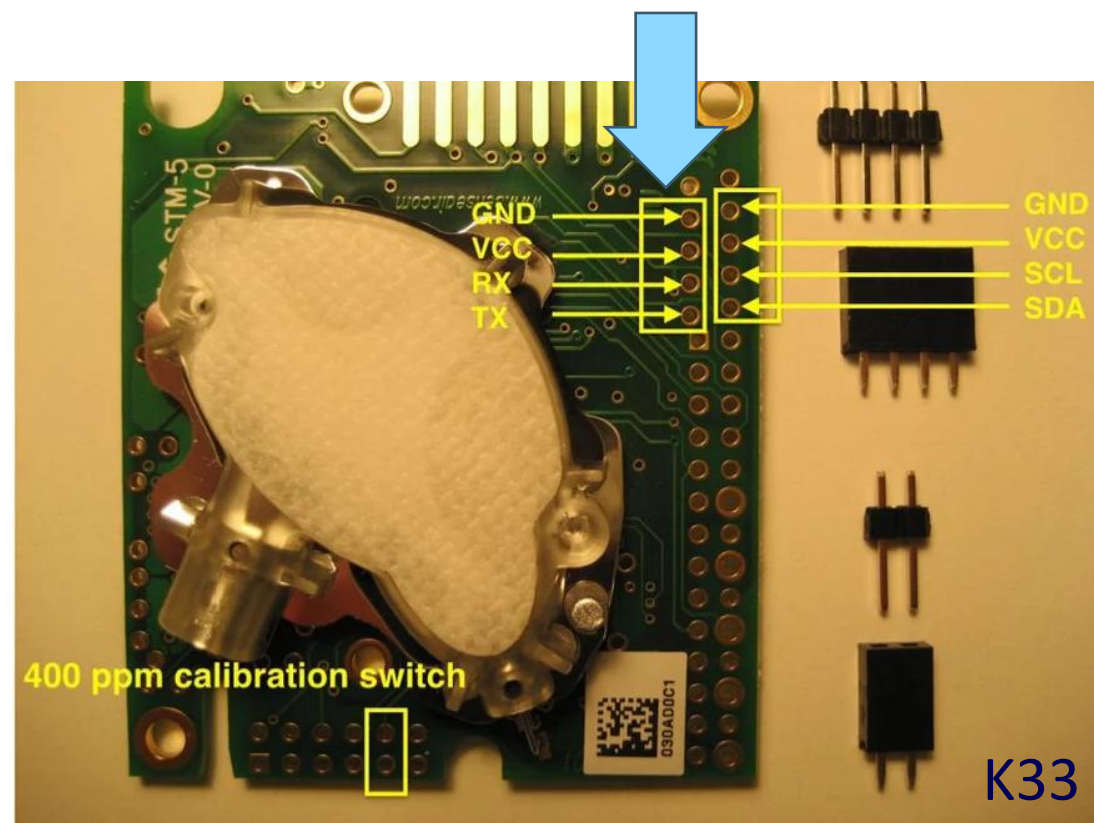
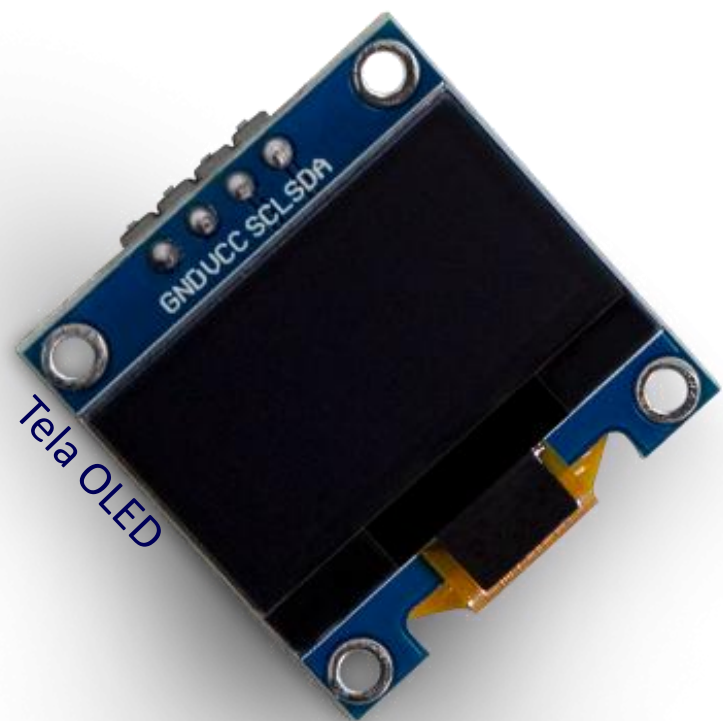
    ...
```



I2C (Inter-integrated circuit, IIC ou I2C)

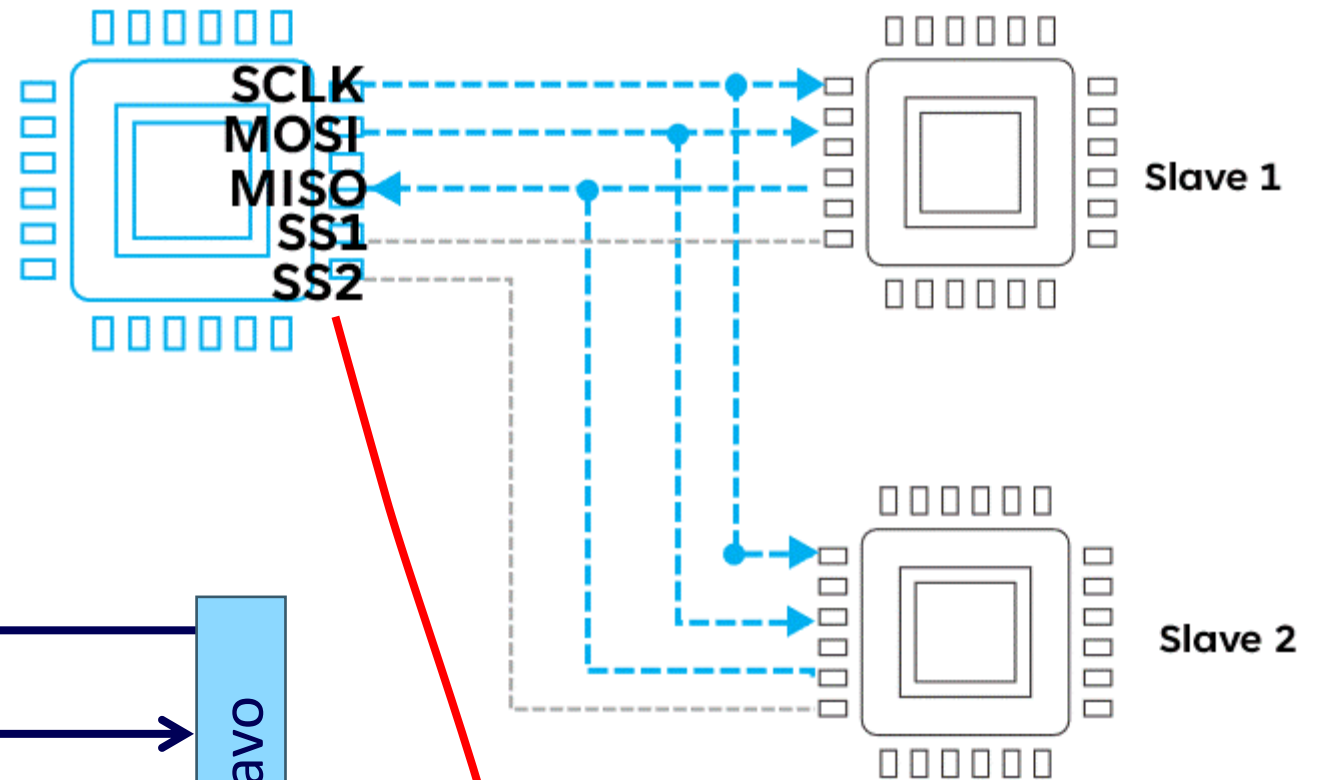
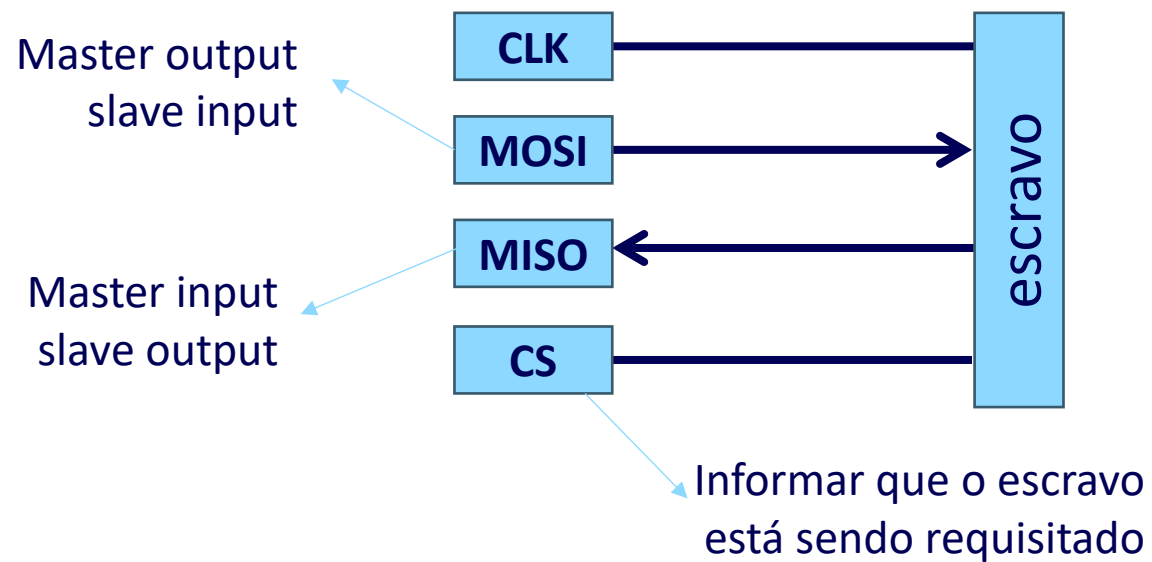
```
Wire.requestFrom(Sensor_Ad, 2);  
  
if (Wire.available() ≥ 2){  
  
    int data1 = Wire.read();  
    data1 = data1<<8;  
  
    in data2 = Wire.read();  
  
    int data = data1|data2;  
  
}  
  
}
```





SPI (Serial Peripheral Interface)

Mais rápido, mas necessita de mais pinos para a comunicação

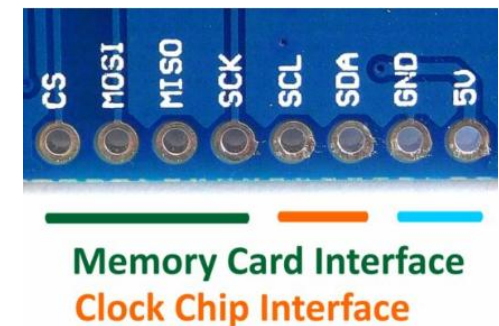
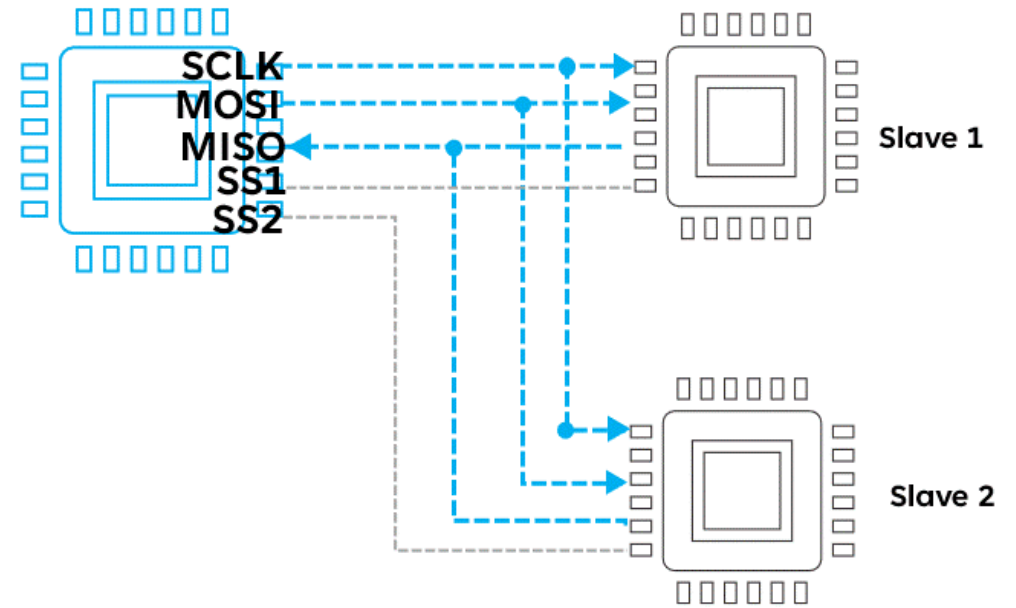


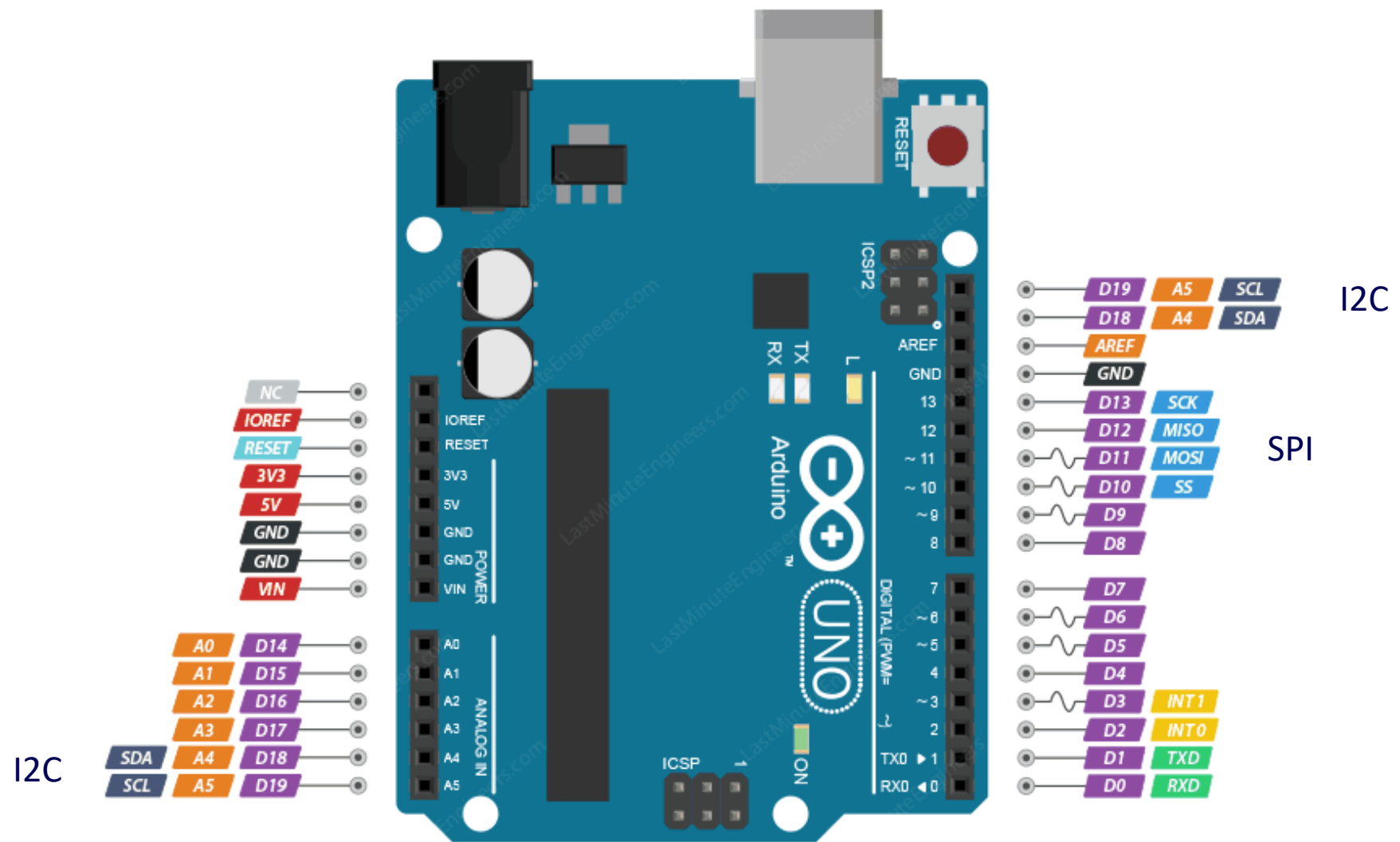
SPI (Serial Peripheral Interface)

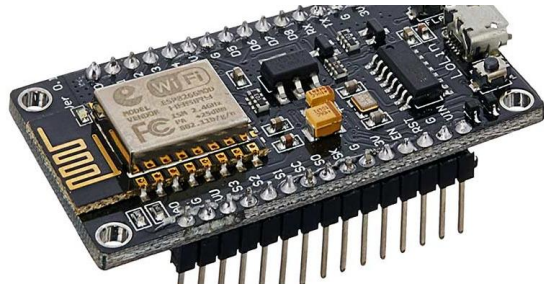
```
#include <SPI.h>
void setup() {
    SPI.begin();
    pinMode(SS_PIN, OUTPUT);
    digitalWrite(SS_PIN, HIGH);
}
void loop(){
    digitalWrite(SS_PIN, LOW);
    byte id = 0x00;
    SPI.transfer(id);

    byte data1 = SPI.transfer(id);
    byte data2 = SPI.transfer(id);
    digitalWrite(SS_PIN, HIGH);

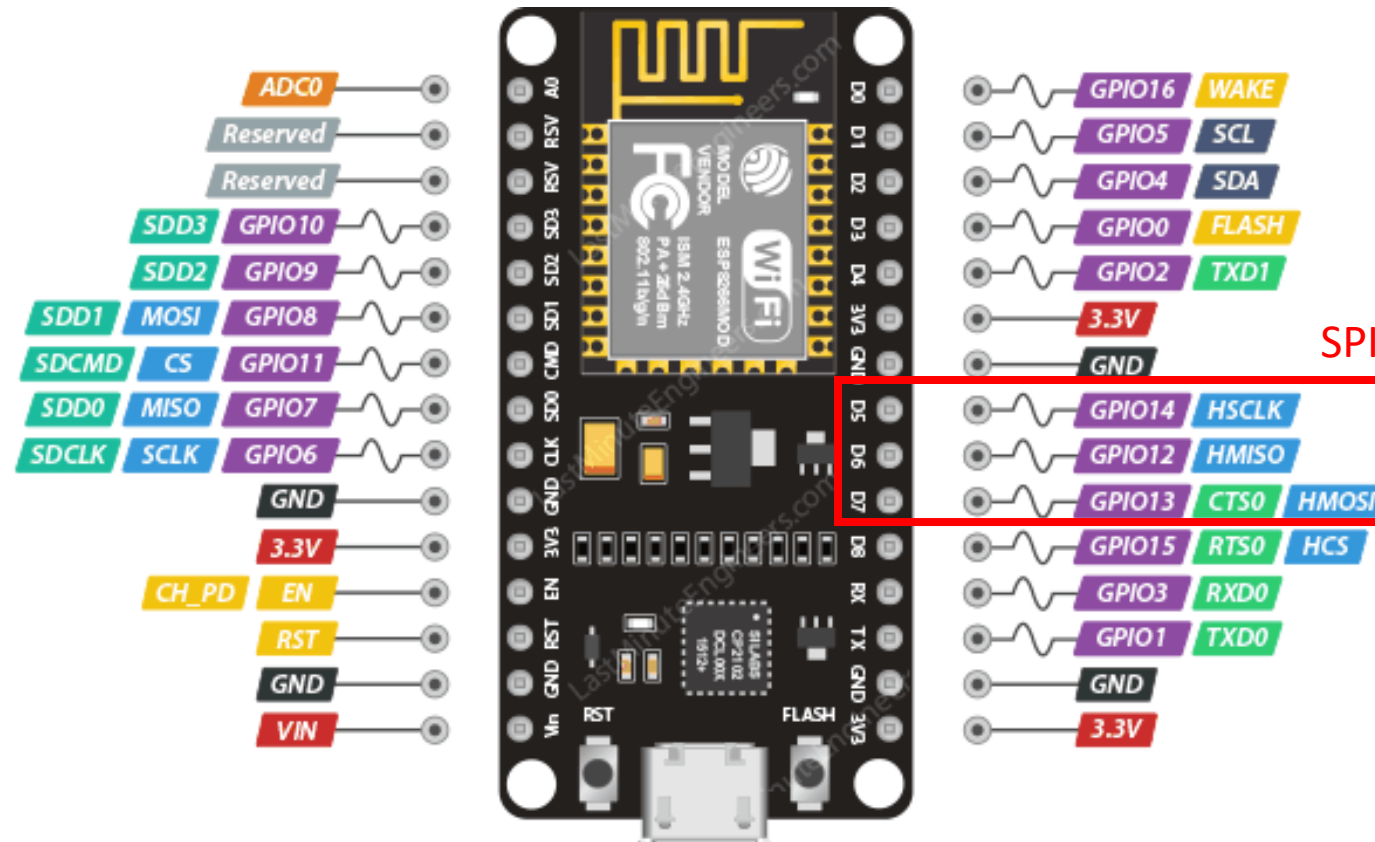
    int data = (data1 << 8) | data2;
```

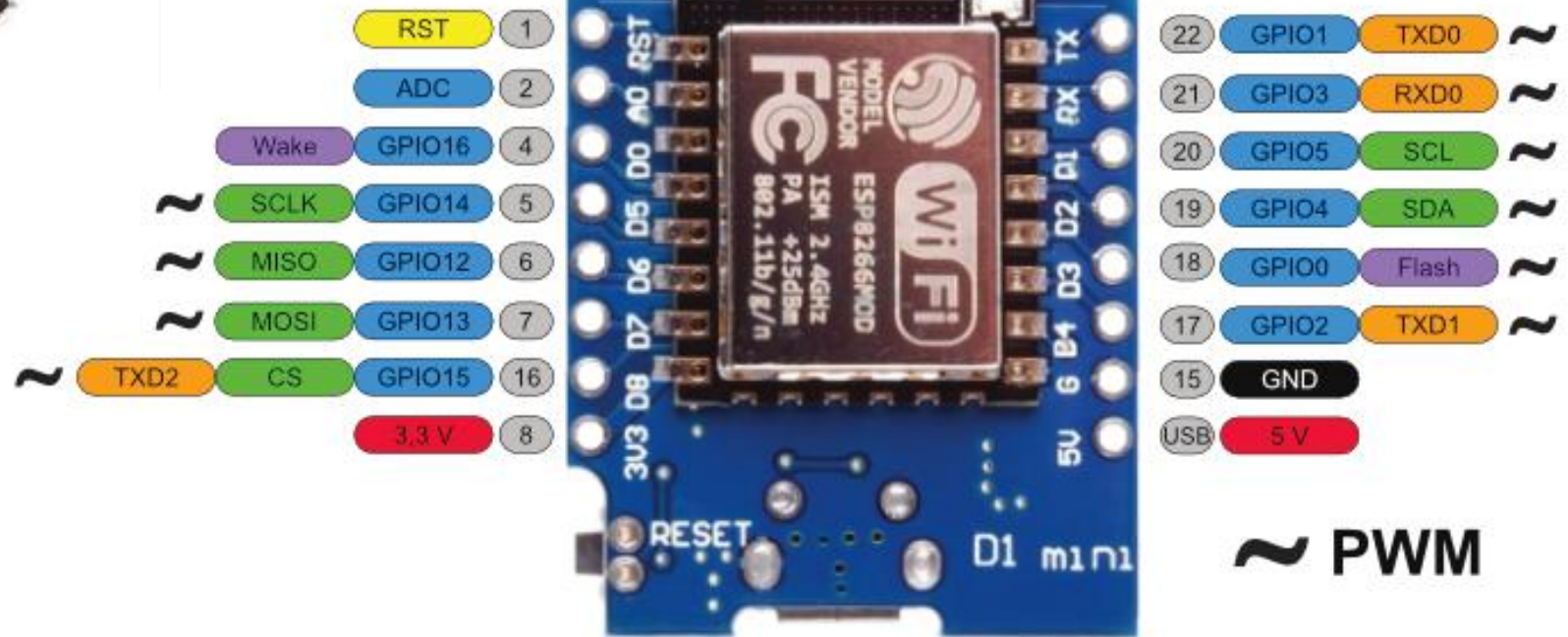






NodeMCU
ESP8266-MOD







Característica		Arduino Uno	Arduino Nano	NodeMCU (ESP8266)	Wemos
Microcontrolador		ATmega328P	ATmega328P	ESP8266	ESP8266
Arquitetura		8-bit	8-bit	32-bit	32-bits
Clock (quantas operações)		16 MHz	16 MHz	80 MHz (160 MHz)	80 MHz (160 MHz)
Memória Flash (HD)		32 KB	32 KB	4000 KB	4000 KB
SRAM (temporários)		2 KB	2 KB	80 KB	80 KB
Tensão de Operação GPIO		5 V	5 V	3.3 V	3.3 V
Tensão de alimentação	USB/Jack	7-12 V	5 V	5 V	5 V
	Vin	7-12 V	7-12 V	5 V	Não existe



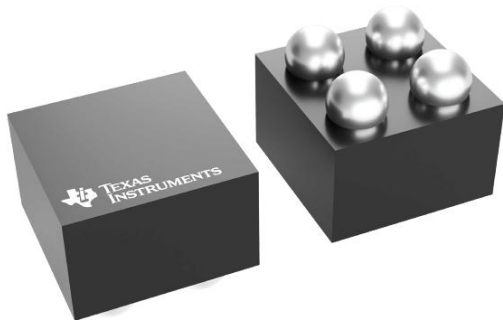
Número de GPIOs	14 D, 6 ADC	14 D, 8 ADC	11 GPIOs	11 GPIOs
Pinos ADC	6 (10 bits, 5 V)	8 (10 bits, 5 V)	1 (10 bits, 3,3 V)	1 (10 bits, 3,3 V)
Saídas PWM	6	6	8	8
Comunicação UART	1	1	2	2
Comunicação I2C	1	1	1	1
Comunicação SPI	1	1	1	1
Conectividade Wi-Fi	Não	Não	Sim	Sim
Bluetooth	Não	Não	Não	Não



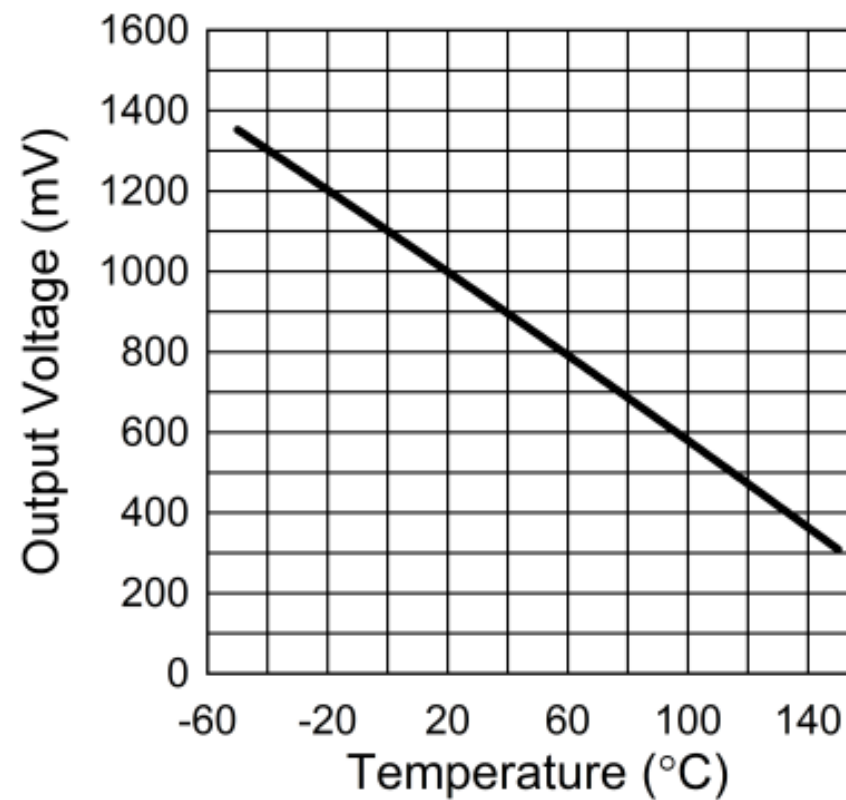
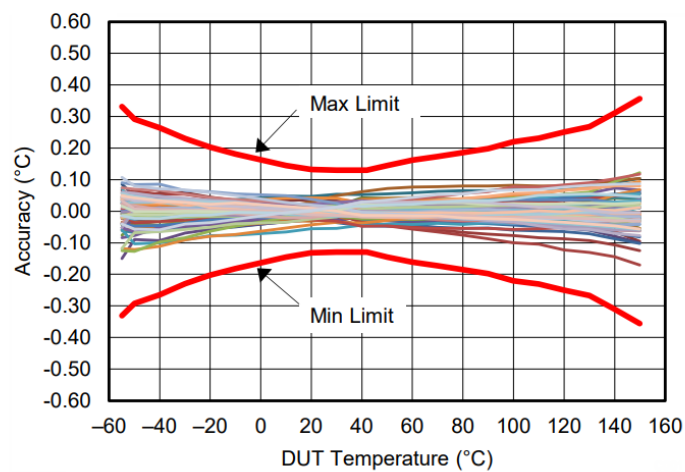
Chega a 300 mA

Consumo de Energia	~ 45 mA	~ 19 mA	~15 mA (80 mA)	~70 mA (80 mA)
Deep Sleep	35 μ A / 30 mA	19 μ A / 15 mA	20 μ A / 9 mA	20 μ A / 170 μ A
Limite de corrente do pino	20 mA	20 mA	12 mA	12 mA

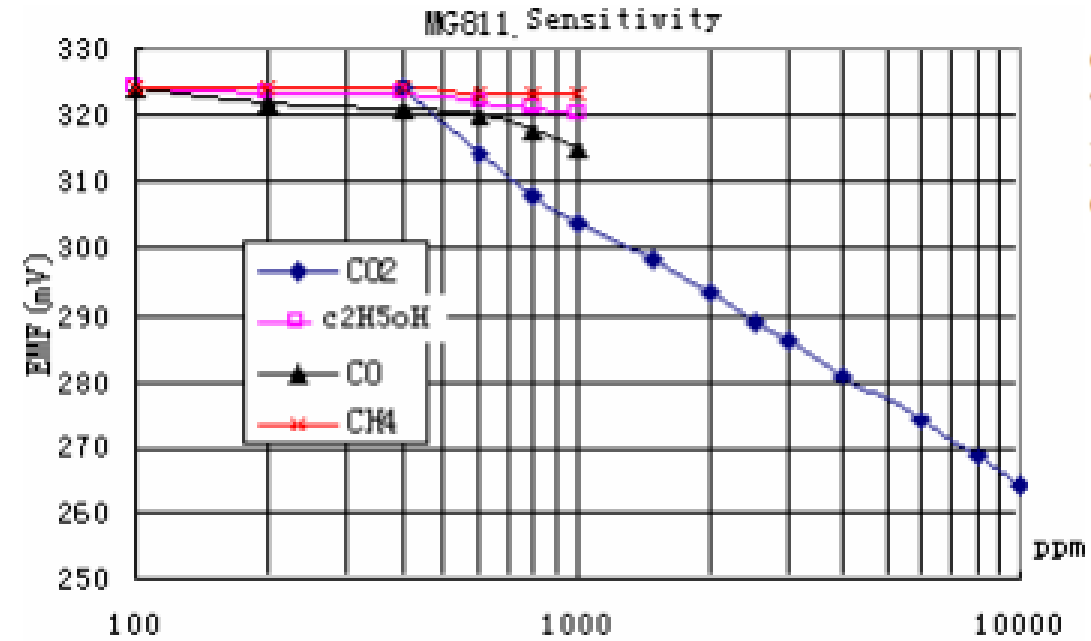
LMT70, LMT70A $\pm 0.05^{\circ}\text{C}$ Precision Analog Temperature Sensor, RTD and Precision NTC Thermistor IC



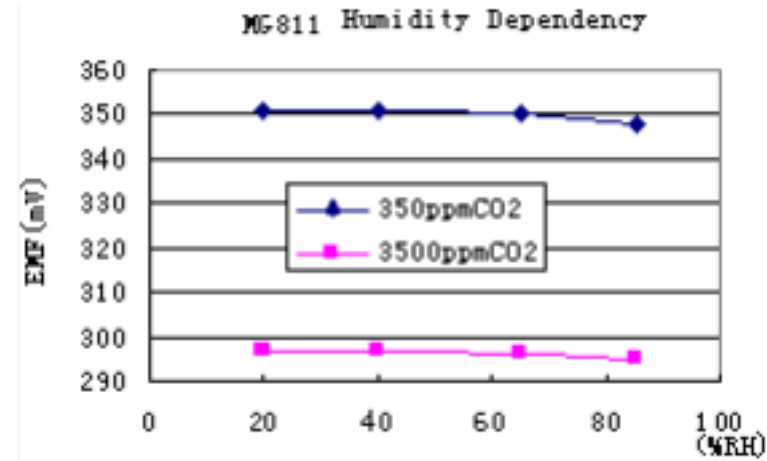
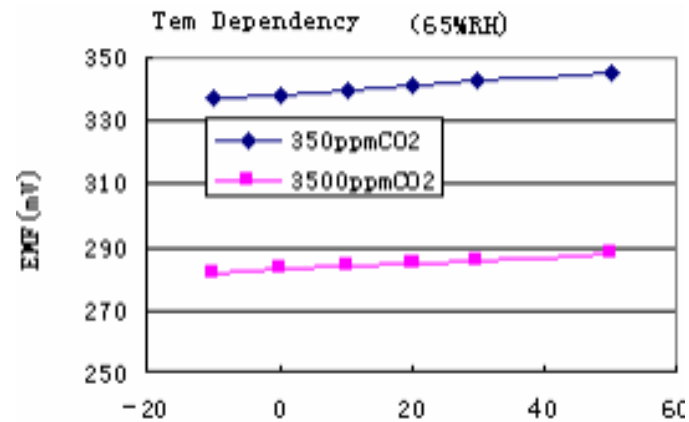
LMT70 Accuracy vs Temperature



MG811 CO2 Sensor



Conditions:
 Tem : 28°C、
 RH: 65%、
 Oxygen : 21%



Exercícios

1. Utilizar o NodeMCU para medir continuamente algum parâmetro e utilizar um LED como sistema de alerta
2. Utilizar ao sistema anterior um leitor de SD que permita salvar os dados coletados
3. Realizar o mesmo sistema mas no Wemos e adicionar um sensor de umidade e temperatura do ar